ESIEE-IT

Projet IA — Utilisation de l'IA

GridLock

Application de suivi des participations étudiantes aux JPO, Salons et Forums

Document Technique & Architecture

US08 · US09 · US10 · US11 · US12 · US13 · US14 · US15 · US16 · US17 · US18 · US19 · US20

Avril 2026

US08 — Écrans clés et règles d'interface

8.1 Mini guide UI — Règles de cohérence visuelle

Ce guide UI s'applique à tous les écrans de GridLock. Il sert de référence aux développeurs et aux agents IA.

Palette de couleurs

Rôle	Couleur hex	Usage
Primaire (Navy)	#0B1F3A	En-têtes, sidebar, textes titres
Accent (Bleu)	#1A56A8	Boutons principaux, liens, focus rings
Gold	#F59E0B	Badges ambassadeur, alertes douces
Succès	#22C55E	Statut Présent, confirmation
Alerte	#F59E0B	Statut Confirmé (en attente)
Erreur	#EF4444	Validation erreur, suppression
Neutre	#F8FAFF / #E8EEF7	Fonds de page et de carte

Typographie

Élément	Font	Taille	Grammage
Titre de page	Syne	1.8rem	800
Titre de section	Syne	1.2rem	700
Corps de texte	Inter	0.9rem	400
Label de formulaire	Inter	0.85rem	500
Badge / tag	Inter	0.7rem	600 / uppercase
Message d'erreur	Inter	0.82rem	400 / rouge

Composants récurrents

- Bouton primaire : fond #1A56A8, texte blanc, border-radius 8px, hauteur 40px, hover opacity 90%
- Bouton secondaire : fond transparent, bordure 1px #1A56A8, texte #1A56A8, même dimensions
- Bouton danger : fond #EF4444, texte blanc — uniquement pour suppression
- Input texte : fond blanc, bordure 1px #E8EEF7, focus ring 2px #1A56A8, hauteur 40px, placeholder gris
- Badge de statut : pills arrondies, couleur selon statut (vert=Présent, or=Confirmé, gris=Absent)
- Badge type d'événement : pills colorées (bleu=Salon, or=JPO, vert=Forum, violet=Évènement)

- Card : fond blanc, bordure 1px #E8EEF7, border-radius 12px, shadow légère
- Toast de confirmation : apparaît en bas droite, disparaît après 3s, vert si succès, rouge si erreur

Navigation

- Sidebar fixe à gauche (240px) sur desktop : logo, items de navigation avec icônes, compte utilisateur en bas
- Items actifs : fond #EBF3FB, bordure gauche 3px #1A56A8, texte #1A56A8
- Sur mobile : hamburger menu, sidebar en overlay
- Breadcrumb présent sur toutes les pages profondes

Formulaires et feedback

- Validation en temps réel sur blur (pas au keystroke)
- Messages d'erreur sous le champ concerné, jamais en popup
- Bouton de soumission désactivé tant que les champs obligatoires sont vides
- État de chargement : spinner sur le bouton, pas de skeleton global

8.2 Écrans structurants — Description des zones et actions

Écran 1 — Dashboard admin (état normal)

Zones	Header (titre page + bouton 'Nouvel événement'), 4 cartes KPI (total événements, participations, présents, ambassadeurs), tableau liste événements avec filtres
Actions disponibles	Filtrer par mois / type / formation | Cliquer sur un événement → fiche événement | Créer un nouvel événement
État vide	Message 'Aucun événement cette année' + bouton CTA 'Créer le premier événement'
État erreur	Banner rouge en haut : 'Impossible de charger les données. Réessayer.'

Écran 2 — Formulaire création/édition événement

Zones	Formulaire en deux colonnes : col gauche (mois, type, date, horaires), col droite (nom structure, nom événement, ville, besoins) — bouton Enregistrer en bas
Actions disponibles	Saisir les champs | Annuler → retour liste | Enregistrer → validation puis redirection fiche événement
État erreur	Champs obligatoires surlignés en rouge, message sous chaque champ invalide

Règle	Le champ 'Type' est une liste déroulante figée : JPO, Salon, Forum, Évènement

Écran 3 — Fiche événement + liste des participants

Zones	En-tête fiche (nom, date, lieu, type, statut), bouton 'Ajouter un participant', tableau des participants avec colonnes Nom/Prénom/Formation/Statut/Ambassadeur/Actions
Actions disponibles	Ajouter participant (modale) | Changer statut (dropdown inline) | Cocher/décocher ambassadeur | Supprimer participant (avec confirmation)
État vide	'Aucun participant inscrit à cet événement' + bouton 'Inscrire un étudiant'
État erreur	Si étudiant déjà inscrit : toast rouge 'Cet étudiant est déjà inscrit à cet événement'

Écran 4 — Récap par étudiant (vue admin)

Zones	Champ de recherche étudiant (auto-complétion), fiche étudiant (nom, formation, email), compteurs (total, par type), liste chronologique des participations
Actions disponibles	Rechercher un étudiant | Filtrer ses participations par type | Cliquer sur un événement → fiche événement
État vide	'Cet étudiant n'a participé à aucun événement cette année'

Écran 5 — Dashboard étudiant (vue personnelle)

Zones	Accueil personnalisé (Bonjour [Prénom]), compteurs de participations de l'année, liste de ses participations avec statuts
Actions disponibles	Filtrer par type d'événement | Voir le détail d'un événement (lecture seule)
État vide	'Vous n'avez pas encore participé à un événement. Contactez votre coordinateur Sourcing.'
Restrictions	L'étudiant ne peut modifier aucune donnée — interface en lecture seule

US09 — Architecture globale et responsabilités

9.1 Vue d'ensemble

GridLock suit une architecture trois-tiers classique : interface utilisateur (React), API REST (Node.js/Express), base de données relationnelle (PostgreSQL). L'ensemble est conteneurisé via Docker Compose pour l'exécution locale et le déploiement.

9.2 Blocs architecturaux

Bloc	Technologie	Responsabilité
Frontend (UI)	React + Vite + TailwindCSS	Affichage, navigation, formulaires, appels API, gestion de l'état UI
API REST (Backend)	Node.js + Express.js	Routage HTTP, validation des entrées, logique métier, authentification JWT
ORM	Prisma	Abstraction BDD, migrations de schéma, queries typées
Base de données	PostgreSQL 15	Persistance des données : événements, étudiants, participations, utilisateurs
Auth	JWT (jsonwebtoken)	Génération et vérification des tokens, middleware de protection des routes
Conteneurisation	Docker + Docker Compose	Isolation des services, reproductibilité de l'environnement

9.3 Flux principaux entre blocs

- Requête utilisateur → Frontend (React) → appel fetch vers API REST → middleware Auth (vérification JWT) → contrôleur → Prisma ORM → PostgreSQL
- Réponse : PostgreSQL → Prisma (résultat typé) → contrôleur (formatage JSON) → API → Frontend (mise à jour état React)
- Authentification : Frontend POST /api/auth/login → vérification BDD → génération JWT → stockage côté client (localStorage) → inclus dans header Authorization de toutes les requêtes

9.4 Limites de l'architecture

- Pas de cache (Redis) dans le MVP — les données sont rechargées à chaque requête
- Pas de WebSocket — les mises à jour multi-utilisateurs nécessitent un refresh manuel
- Pas de CDN ni de load balancer — architecture single-instance adaptée au contexte étudiant

US10 — Choix techniques et justification de stack

10.1 Critères de sélection

Critère	Description	Poids
Rapidité d'apprentissage	La stack doit être maîtrisable par l'équipe étudiante en temps contraint	Élevé
Maintenabilité	Code lisible, conventions claires, ORM pour éviter le SQL brut error-prone	Élevé
Testabilité	Support Jest/Vitest, injection de dépendances, mocking facilité	Moyen
Déploiement	Containerisable facilement, pas de dépendance cloud externe obligatoire	Élevé
Sécurité	Gestion JWT native, Prisma protège des injections SQL, validation entrées	Élevé
Écosystème	Packages bien maintenus, documentation riche, communauté active	Moyen

10.2 Choix retenus vs. alternatives

Composant	Retenu	Alternative 1	Alternative 2	Justification du choix
Frontend	React + Vite	Vue 3	Angular	Courbe d'apprentissage plus courte pour l'équipe, écosystème plus large, Vite pour la rapidité de build
Backend	Node.js / Express	Python / FastAPI	PHP / Laravel	JavaScript isomorphe front+back, Express ultra-léger pour le MVP, maîtrisé par l'équipe
ORM	Prisma	Sequelize	TypeORM	Schéma déclaratif, migrations automatiques, types générés, meilleure DX pour étudiant
BDD	PostgreSQL	MySQL	SQLite	Relationnel robuste, support JSON natif, Docker officiel, pas de limitation de production comme SQLite
Auth	JWT	Sessions + cookies	Passeport.js	Stateless = facilite le déploiement, simple à implémenter, suffisant pour le MVP
CSS	TailwindCSS	SCSS modules	Styled-components	Utility-first = cohérence forcée, pas de CSS global, rapide à prototyper

10.3 Impacts sur la qualité, sécurité et CI/CD

- Qualité : Prisma impose des types sur les modèles de données, réduisant les bugs de type runtime. ESLint + Prettier sont configurés sur le projet.

- Sécurité : Prisma protège nativement contre les injections SQL. Les tokens JWT expirent (configurable, défaut 24h). Les variables sensibles passent par .env non commité.
- CI/CD : Jest (backend) + Vitest (frontend) sont directement intégrables dans GitHub Actions. Docker facilite les vérifications de build dans la pipeline.

US11 — Diagramme de séquence du parcours principal

Le diagramme de séquence est fourni en annexe visuelle (fichier SVG/HTML). La description textuelle ci-dessous sert de référence pour les développeurs.

11.1 Parcours principal — Inscription d'un étudiant à un événement

Acteurs : Sophie (Coordinatrice Admin), Frontend React, API Express, PostgreSQL

#	Acteur source	Message / Action	Acteur cible	Validation / Résultat
1	Sophie	Clique 'Ajouter un participant' sur la fiche événement	Frontend	Ouverture de la modale d'inscription
2	Frontend	GET /api/students?q={nom} (auto-complétion)	API	Validation token JWT, paramètre q requis
3	API	SELECT * FROM students WHERE nom ILIKE '%{q}%'	PostgreSQL	Retour liste étudiants correspondants
4	PostgreSQL	Résultat : liste étudiants	API → Frontend	Affichage des suggestions dans le champ
5	Sophie	Sélectionne un étudiant, choisit statut 'Confirmé'	Frontend	Formulaire validé côté client
6	Frontend	POST /api/events/{id}/participations {studentId, statut}	API	Vérification JWT + rôle admin
7	API	Vérifie que l'étudiant n'est pas déjà inscrit	PostgreSQL	SELECT COUNT(*) WHERE eventId AND studentId
8a (nominal)	API	INSERT INTO participations	PostgreSQL	Participation créée, retour 201 Created
8b (erreur)	API	Étudiant déjà inscrit → 409 Conflict	Frontend	Toast erreur 'Déjà inscrit'
9	Frontend	Ferme la modale, met à jour la liste des participants	Sophie	Confirmation visuelle : toast vert

11.2 Cas d'erreur représentatif — Token expiré

- Si le JWT de Sophie a expiré pendant sa session, toute requête API retourne 401 Unauthorized.
- Le frontend intercepte ce code et redirige automatiquement vers /login avec un message 'Session expirée'.

- Après reconnexion, l'utilisateur est redirigé vers la page qu'il consultait (paramètre redirect dans l'URL).

US12 — Modèle conceptuel Objet et Modèle de données

12.1 Entités principales et relations

Le modèle de données est fourni en diagramme de classe visuel (fichier SVG/HTML). Description textuelle ci-dessous.

Entité	Description	Relations
User	Compte d'accès à l'application (admin, étudiant, directeur)	1 User peut être lié à 0 ou 1 Student (rôle étudiant)
Student	Apprenant référencé dans la base ESIEE-IT	1 Student → N Participations | 0..1 User
Event	Événement de promotion (JPO, Salon, Forum, Évènement)	1 Event → N Participations
Participation	Lien entre un Event et un Student avec statut et rôle ambassadeur	N..1 Event, N..1 Student

12.2 Schéma de données détaillé (PostgreSQL)

Table : users

Champ	Type	Contraintes	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Identifiant unique
email	VARCHAR(255)	NOT NULL, UNIQUE	Email de connexion (email école)
password_hash	VARCHAR(255)	NOT NULL	Mot de passe hashé (bcrypt, 12 rounds)
role	ENUM('admin','student','director')	NOT NULL, DEFAULT 'student'	Rôle applicatif
student_id	UUID	FK → students.id, NULLABLE	Lien optionnel vers un apprenant
created_at	TIMESTAMP	DEFAULT NOW()	Date de création du compte

Table : students

Champ	Type	Contraintes	Description
id	UUID	PK	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom de famille (MAJUSCULES)
prenom	VARCHAR(100)	NOT NULL	Prénom

email	VARCHAR(255)	UNIQUE, NULLABLE	Email école (format edu.esiee-it.fr)
telephone	VARCHAR(20)	NULLABLE	Numéro de téléphone
formation	VARCHAR(150)	NOT NULL	Intitulé de la formation (ex: Coding Cergy - DSNS - B3)
annee	VARCHAR(50)	NULLABLE	Année de formation (ex: 2ème année)
etab_origine	VARCHAR(200)	NULLABLE	Établissement d'origine avant ESIEE-IT
created_at	TIMESTAMP	DEFAULT NOW()	Date d'import

Table : events

Champ	Type	Contraintes	Description
id	UUID	PK	Identifiant unique
mois	VARCHAR(20)	NOT NULL	Mois de l'événement (ex: Octobre)
type	ENUM('JPO','Salon','Forum','Evènement')	NOT NULL	Type d'événement
date_event	DATE	NOT NULL	Date de l'événement
nom_structure	VARCHAR(300)	NOT NULL	Nom de la structure accueillante
nom_evenement	VARCHAR(300)	NOT NULL	Nom de l'événement
ville	VARCHAR(100)	NOT NULL	Ville
horaires	VARCHAR(100)	NULLABLE	Créneaux horaires (ex: 10h - 18h)
besoins	VARCHAR(300)	NULLABLE	Besoins en profils (ex: 1 étudiant Coding)
created_by	UUID	FK → users.id	Créateur de la fiche
created_at / updated_at	TIMESTAMP	DEFAULT NOW()	Horodatages

Table : participations

Champ	Type	Contraintes	Description
id	UUID	PK	Identifiant unique
event_id	UUID	FK → events.id, NOT NULL, ON	Événement concerné

		DELETE CASCADE	
student_id	UUID	FK → students.id, NOT NULL	Étudiant inscrit
statut	ENUM('confirmé','présent','absent')	NOT NULL, DEFAULT 'confirmé'	Statut de participation
est_ambassadeur	BOOLEAN	NOT NULL, DEFAULT FALSE	Rôle ambassadeur
confirme_par	UUID	FK → users.id, NULLABLE	Admin ayant confirmé
created_at / updated_at	TIMESTAMP	DEFAULT NOW()	Horodatages
UNIQUE	(event_id, student_id)	Contrainte d'unicité	Un étudiant ne peut être inscrit qu'une fois par événement

US13 — Rôles, permissions et gestion des erreurs

13.1 Matrice des rôles et permissions

Action	Admin	Director	Student
Voir la liste des événements	✓	✓ (lecture)	✗
Créer / modifier / supprimer un événement	✓	✗	✗
Ajouter un participant à un événement	✓	✗	✗
Modifier le statut d'un participant	✓	✗	✗
Supprimer un participant d'un événement	✓	✗	✗
Voir le récap par étudiant (tous étudiants)	✓	✓	✗
Voir son propre récap de participations	✓	✓	✓
Accéder au tableau de bord KPI global	✓	✓	✗
Gérer les comptes utilisateurs	✓	✗	✗
Importer la base apprenants	✓	✗	✗

13.2 Gestion des erreurs — Taxonomie

Erreurs de validation (400 Bad Request)

- Champ obligatoire manquant : 'Le champ [nom] est requis.'
- Format invalide : 'L'email doit être au format nom@edu.esiee-it.fr.'
- Valeur hors liste autorisée : 'Le type d'événement doit être JPO, Salon, Forum ou Évènement.'

Erreurs d'autorisation (401 / 403)

- 401 Unauthorized : token absent ou expiré → redirection /login
- 403 Forbidden : token valide mais rôle insuffisant → page 403 avec message 'Vous n'avez pas les droits pour effectuer cette action.'

Erreurs métier (409 / 404)

- 409 Conflict : tentative d'inscription d'un étudiant déjà inscrit → toast erreur côté frontend
- 404 Not Found : ressource inexistante (événement supprimé) → page 404 avec bouton retour

Erreurs techniques (500)

- Toute erreur serveur non anticipée retourne 500 avec un message générique : 'Une erreur est survenue. Veuillez réessayer ou contacter l'administrateur.'
- Les détails techniques (stack trace) ne sont jamais exposés au client — ils sont loggés côté serveur uniquement.

US14 — Contrat des routes API

Authentification

Base URL	/api
Auth requise	Header Authorization: Bearer {jwt_token} sur toutes les routes sauf /auth/login
Format	JSON exclusivement — Content-Type: application/json

Routes : Authentification

Méthode	Route	Corps / Params	Réponse succès	Erreurs
POST	/auth/login	{email, password}	200 {token, user: {id, role, email}}	400 champs manquants | 401 identifiants invalides
POST	/auth/logout	Aucun	200 {message: 'Déconnecté'}	401 token absent

Routes : Événements

Méthode	Route	Corps / Params	Réponse succès	Erreurs
GET	/events	?type=&mois=&formation=	200 [{id, type, date, nom...}]	401 | 403
POST	/events	{type, date, nom_structure, nom_evenement, ville, mois}	201 {event créé}	400 validation | 401 | 403 (admin only)
GET	/events/:id	—	200 {event + participations[]}	401 | 404
PUT	/events/:id	{champs modifiés}	200 {event mis à jour}	400 | 401 | 403 | 404
DELETE	/events/:id	—	204 No Content	401 | 403 | 404

Routes : Participations

Méthode	Route	Corps / Params	Réponse succès	Erreurs
GET	/events/:id/participations	—	200 [{participation + student}]	401 | 404
POST	/events/:id/participations	{student_id, statut, est_ambassadeur}	201 {participation créée}	400 | 401 | 403 | 409 déjà inscrit

PUT	/participations/:id	{statut?, est_ambassadeur?}	200 {participation mise à jour}	400 | 401 | 403 | 404
DELETE	/participations/:id	—	204 No Content	401 | 403 | 404

Routes : Étudiants

Méthode	Route	Corps / Params	Réponse succès	Erreurs
GET	/students	?q= (auto-complétion, min 2 chars)	200 [{id, nom, prenom, formation}]	401 | 403 (admin/director)
GET	/students/:id	—	200 {student complet}	401 | 403 | 404
GET	/students/:id/participations	—	200 [{participation + event}]	401 | 403 | 404
GET	/students/me/participations	Token étudiant	200 [{participations de l'étudiant connecté}]	401

Routes : Dashboard

Méthode	Route	Réponse succès	Erreurs
GET	/dashboard	200 {total_events, total_participations, total_presents, total_ambassadeurs, by_type[]}	401 | 403 (admin/director)

US15 — Journal des décisions produit et techniques

Date	Contexte	Options envisagées	Décision retenue	Compromis accepté
Avr. 2026	Choix du nom de l'application	EventTrack (nom initial) vs GridLock	GridLock — nom retenu par l'équipe projet	Aucun impact technique — renommage dans le code et la doc
Avr. 2026	Choix de la base de données	PostgreSQL vs MySQL vs SQLite	PostgreSQL — robustesse, support UUID natif, Docker officiel	Configuration légèrement plus lourde que SQLite, mais aucune limite de production
Avr. 2026	Stratégie d'authentification	JWT vs sessions côté serveur vs Passport.js	JWT — stateless, simple à implémenter, suffisant pour le MVP	Pas de révocation immédiate possible (le token reste valide jusqu'à expiration)
Avr. 2026	ORM vs SQL brut	Prisma vs Sequelize vs SQL brut	Prisma — meilleure DX, migrations déclaratives, types générés	Abstraction supplémentaire : certaines requêtes optimisées nécessiteront du SQL brut
Avr. 2026	Hors périmètre : envoi d'emails	Intégrer nodemailer vs ne pas envoyer d'email	Pas d'envoi d'email dans le MVP — complexité SMTP non justifiée	Les coordinateurs doivent notifier manuellement les étudiants confirmés
Avr. 2026	Import de la base apprenants	Import CSV automatique vs saisie manuelle vs copie depuis l'Excel	Import CSV one-shot via script de seed Prisma (migration initiale)	Pas d'interface d'import dans le MVP — à prévoir en V2
Avr. 2026	Modèle de déploiement	Cloud (Render/Railway) vs hébergement interne Docker	Docker Compose en local / interne — souveraineté des données	Pas d'URL publique pour la démo — démo locale ou sur réseau interne

US16 — Stratégie Git et règles d'intégration

16.1 Structure des branches

Branche	Rôle	Règle
main	Branche de référence — code stable et testé uniquement	Aucun commit direct interdit. Intégration uniquement via Pull Request validée.
develop	Branche d'intégration continue	Les features sont mergées ici après revue. Sert de base pour les nouvelles branches.
feature/{us-id}-{description}	Développement d'une fonctionnalité	Ex: feature/us23-inscription-participant. Branchée depuis develop.
fix/{description}	Correction de bug	Ex: fix/statut-ambassadeur-null. Branchée depuis develop ou main si hotfix.
chore/{description}	Tâche technique (config, CI, deps)	Ex: chore/setup-docker-compose. Pas de logique métier.

16.2 Règles de fusion

- Toute intégration vers develop ou main passe par une Pull Request avec au moins une revue (auto-revue documentée si solo).
- La PR doit passer tous les checks CI (tests, lint, build) avant de pouvoir être mergée.
- Le merge squash est recommandé vers develop pour garder un historique lisible.
- La branche main ne reçoit que des PRs depuis develop, après validation de l'environnement de développement.

16.3 Protection de main

- Aucun push direct sur main — protection de branche activée sur GitHub.
- Statut checks obligatoires : tests unitaires + lint.

US17 — Convention de commits et découpage atomique

17.1 Format des commits

```
<type>(<scope>): <description courte en impératif>

[Corps optionnel — explication du pourquoi si non évident]

[Footer optionnel — références issues, breaking changes]
```

17.2 Types autorisés

Type	Usage	Exemple
feat	Nouvelle fonctionnalité liée à une US	feat(participation): add student registration to event
fix	Correction de bug	fix(auth): handle expired token redirect
test	Ajout ou modification de tests	test(participation): add unit test for duplicate check
chore	Tâche technique sans logique métier	chore(docker): add compose file for local dev
docs	Documentation uniquement	docs(api): document /events routes contract
refactor	Refactorisation sans changement de comportement	refactor(events): extract validation to middleware
style	Formatage, lint uniquement	style: run prettier on backend files
ci	Configuration pipeline CI/CD	ci: add GitHub Actions workflow for tests

17.3 Règles de découpage atomique

- Un commit = une fonctionnalité, une correction ou une tâche — jamais plusieurs.
- Si un commit embarque du code nouveau, les tests associés doivent être dans le même commit (ou le commit suivant immédiat avec justification).
- Les fichiers de configuration (.env.example, docker-compose.yml) ne sont pas mélangés avec du code fonctionnel.
- Un commit WIP (Work In Progress) est interdit sur les branches develop et main.

17.4 Exemples acceptés / refusés

Commit	Statut	Raison
feat(events): implement CRUD endpoints for events	ACCEPTÉ	Un seul sujet, description claire
fix(auth): redirect to login on 401	ACCEPTÉ	Correction précise et atomique

ajout de trucs et fix de bugs	REFUSÉ	Pas de type, pas de scope, vague
feat: add events + fix participation bug + update styles	REFUSÉ	3 sujets différents dans 1 commit
wip	REFUSÉ	Non descriptif, interdit sur develop/main

US18 — Configuration Antigravity

La configuration Antigravity ci-dessous encadre le comportement des agents IA sur le projet GridLock. Elle est à placer à la racine du projet (antigravity.config.md ou .antigravity).

```
# GridLock — Configuration Antigravity
# Projet ESIEE-IT — Utilisation de l'IA — Avril 2026

## PÉRIMÈTRE
- L'agent ne modifie QUE les fichiers liés à la US en cours de traitement
- Tout changement hors scope (autre US, autre module) est interdit sans validation
humaine explicite
- Avant tout changement structurel (nouveau service, nouvelle table, nouveau
composant majeur),
  l'agent formule une proposition et attend une validation humaine

## VÉRIFICATIONS OBLIGATOIRES AVANT COMMIT
- [ ] Les tests unitaires et d'intégration passent
- [ ] ESLint / Prettier ne remonte aucune erreur bloquante
- [ ] Aucun secret (mot de passe, clé API) n'est présent dans le code
- [ ] La documentation concernée par la US est mise à jour
- [ ] Le commit suit la convention définie (US17)

## RÈGLES GIT
- Aucun push direct sur main ou develop
- Chaque livraison = une branche feature/{us-id}-{description}
- Commits atomiques : 1 sujet = 1 commit
- Format : <type>(<scope>): <description>

## SÉCURITÉ
- Aucun secret hardcodé — utiliser process.env + .env.example
- Prisma protège des injections SQL — ne pas contourner avec des requêtes brutes
non validées
- Les mots de passe sont hashés avec bcrypt (min 12 rounds) — jamais en clair
- Le rôle de l'utilisateur est vérifié côté API, jamais seulement côté frontend

## TRAÇABILITÉ
- Toute décision structurante est ajoutée au journal des décisions (US15)
- Les modifications d'architecture nécessitent une mise à jour du schéma Prisma ET
du document US12
- Tout ajout de dépendance doit être justifié dans le commit et dans la SBOM
(US33)
```

US19 — Règles de travail communes des agents IA

19.1 Comportement attendu des agents

- L'agent explicite ses hypothèses avant de coder : 'Je suppose que X — confirmes-tu ?'
- L'agent annonce les fichiers qu'il va modifier avant d'agir : 'Je vais modifier src/api/events.js et src/routes/events.routes.js'
- L'agent signale les points incertains plutôt que de deviner en silence
- L'agent refuse les demandes qui changeraient le périmètre, la sécurité ou l'architecture sans validation humaine

19.2 Règles par type de production

Type	Règle
Génération de code	Suivre ESLint, Prettier, conventions de nommage du projet. Pas de console.log en production.
Documentation	Mise à jour du fichier concerné uniquement. Pas de refonte globale non demandée.
Refactoring	Zéro changement de comportement. Tests avant et après identiques. PR dédiée.
Tests	Couvrir le cas nominal ET au moins un cas d'erreur. Mocks pour la BDD en tests unitaires.
Validation	L'agent ne valide pas lui-même son propre code — il signale ce qui reste à vérifier.

US20 — Garde-fous qualité et sécurité des agents

20.1 Contrôles obligatoires avant acceptation d'une production IA

- Relecture humaine de tout code proposé contre les critères : lisibilité, responsabilité unique, pas de duplication
- Vérification absence de secrets : l'agent scanne son propre output pour toute clé, mot de passe ou URL sensible hardcodée
- Vérification des droits : toute route API dans le code généré doit avoir un middleware d'authentification et de vérification du rôle
- Vérification des dépendances : tout nouveau import doit être justifié et absent de la liste des packages inutiles
- Signal de dette technique : l'agent documente explicitement les raccourcis pris ('TODO : ajouter validation X')

20.2 Alignement SonarQube, tests et Git

- SonarQube : l'agent produit du code qui respecte les règles de qualité statique — pas de code dupliqué, complexité cyclomatique < 10, pas de code mort
- Tests : tout ajout de règle métier = test associé dans le même commit ou justification documentée
- Git : l'agent ne produit pas de code mêlant plusieurs fonctionnalités — il découpe ses propositions en commits atomiques
- Régressions : l'agent signale tout risque de régression avant de modifier du code existant